

Auto attendance System (ASP Report)



Project Report

Automatic Attendance System using Face Recognition (ASP.NET Edition)

1) Abstract

This project proposes an **Automatic Attendance System** implemented as an **ASP.NET Core web application**. Users (faculty) upload a classroom image or capture from webcam; the system **detects and recognizes** student faces against a pre-enrolled gallery and **logs attendance** to a database.

Two implementation variants are presented:

- **A. Pure .NET:** Face detection & recognition in C# using **OpenCvSharp** and **ArcFace (ONNX)** with cosine similarity.
- **B. Hybrid:** ASP.NET handles UI/DB; recognition is delegated to an existing **Python (DeepFace)** microservice to reuse your working code with minimal changes.

Both produce annotated images, a timestamped attendance record, and summary reports.

2) Introduction

Manual attendance is slow, error-prone, and susceptible to proxy. Modern computer vision enables **automated, reliable** attendance by recognizing faces from a class photo (or frames from a short clip). ASP.NET enables a **browser-based workflow**—easy for faculty to use, centrally managed, and scalable.

3) Problem Statement

- Roll calls and paper sheets waste time and are inaccurate.
 - Proxy/impersonation reduces integrity.
 - Need a **fast**, **auditable**, and **scalable** solution that integrates with institutional systems (ERP/LMS).
-

4) Objectives

1. Build a **web-based** attendance system with upload/capture → detect → recognize → log.
 2. Provide **two tech paths** to fit different teams/infrastructure.
 3. Store attendance in a **relational database** with **evidence** (annotated image, confidence).
 4. Offer **export/reporting** and an admin panel for face enrollment.
-

5) System Requirements

Hardware

- Server/PC for hosting (8 GB RAM recommended)
- Classroom camera or still images from a faculty device

Software

- **ASP.NET Core 8.0+** (Web App/MVC or Minimal API)
 - **SQL Server** (Express is fine)
 - **NuGet (variant A):** `OpenCvSharp4` , `OpenCvSharp4.runtime.win` , `Microsoft.ML.OnnxRuntime` , `Microsoft.ML.OnnxRuntime.Managed`
 - **Python service (variant B):** Your **DeepFace** environment (reusing your code), `FastAPI` / `Flask`
-

6) System Design & Architecture

High-level flow

1. Faculty signs in → navigates to **Attendance** page.
2. Upload classroom image (or capture via webcam) → server receives file.
3. Server **detects faces**, generates embeddings, and **matches** against enrolled gallery.
4. For each match above threshold → **mark attendance** with time, date, confidence.
5. Save the **annotated image** and write an audit log.
6. Display results and allow **CSV/PDF export**.

[Insert Architecture Diagram Placeholder]

(Boxes: Browser → ASP.NET App → (A) .NET Face Service OR (B) Python Face API → SQL Server → Storage for outputs)

Data model (core tables)

- **Students**(StudentId, Name, RollNo, Dept, ...)
- **Faces**(FaceId, StudentId, ImagePath, EmbeddingVector/Blob [A], AddedOn)
- **Attendance**(AttendancId, StudentId, Date, Time, Confidence, SourceImagePath, SessionId)
- **Sessions**(SessionId, Course, FacultyId, Date, Notes)

In variant B, you may store only image paths for gallery and let Python compute embeddings at runtime or precompute & store them.

7) Implementation

7.1 Common ASP.NET Features (for both variants)

- **Pages/Views:**
 - **/Attendance/Upload** – upload or capture image

- **/Attendance/Result** – show boxes, labels, confidence, and a link to annotated image
- **/Students/Enroll** – upload a face per student (1–3 images recommended)
- **/Reports/Daily** – filter by class/date, export CSV/PDF
- **Storage:**
 - `wwwroot/faces/` → enrolled faces (e.g., `wwwroot/faces/Vinay.jpg`)
 - `wwwroot/outputs/` → annotated outputs (`recognized_YYYYMMDD_HHMMSS.jpg`)
- **Security:**
 - Faculty login (ASP.NET Identity)
 - Role: Admin (manage students), Faculty (mark attendance)

 **[Screenshot Placeholder: Enrollment page]**

 **[Screenshot Placeholder: Upload/Capture page]**

7.2 Variant A — Pure .NET (OpenCvSharp + ArcFace ONNX)

Why choose this: single-runtime deployment, no Python dependency.

Key components

- **Face detection:** OpenCv DNN or Haar/SSD/MTCNN (via OpenCvSharp or a small ONNX detector).
- **Embedding model:** ArcFace (ONNX).
- **Similarity:** Cosine distance (lower is more similar); configurable threshold (e.g., 0.40–0.60 depending on scale).

NuGet

```
OpenCvSharp4
OpenCvSharp4.runtime.win
Microsoft.ML.OnnxRuntime
Microsoft.ML.OnnxRuntime.Managed
```

Cosine utility (C#)

```

public static class Similarity {
    public static float Cosine(float[] a, float[] b) {
        double dot = 0, na = 0, nb = 0;
        for (int i = 0; i < a.Length; i++) {
            dot += a[i] * b[i];
            na += a[i] * a[i];
            nb += b[i] * b[i];
        }
        var denom = Math.Sqrt(na) * Math.Sqrt(nb) + 1e-12;
        return (float)(1.0 - (dot / denom)); // distance (lower = better)
    }
    public static float Confidence(float dist, float thr) {
        var conf = 1.0 - (dist / (thr + 1e-6));
        return (float)Math.Clamp(conf, 0, 1);
    }
}

```

ONNX inference skeleton

```

using Microsoft.ML.OnnxRuntime;
using Microsoft.ML.OnnxRuntime.Tensors;

public sealed class FaceEmbedder {
    private readonly InferenceSession _session;
    public FaceEmbedder(string onnxPath) ⇒ _session = new InferenceSession
(onnxPath);

    public float[] Embed(Mat faceAligned) {
        // 1) Preprocess: resize, normalize; 2) to tensor; 3) run session; 4) read o
        utput
        // (Exact preprocess depends on your ArcFace model; align & crop to 112
        x112 typical)
        var input = new DenseTensor<float>(new[] {1, 3, 112, 112});
        // ... fill input tensor ...
        var inputs = new List<NamedOnnxValue> {

```

```

        NamedOnnxValue.CreateFromTensor("input", input)
    };
    using var results = _session.Run(inputs);
    var output = results.First().AsEnumerable<float>().ToArray();
    // L2 normalize
    var norm = Math.Sqrt(output.Sum(v => v*v)) + 1e-12;
    for (int i = 0; i < output.Length; i++) output[i] = (float)(output[i] / norm);
    return output;
}
}

```

Face matching workflow

```

public class RecognitionService {
    private readonly FaceEmbedder _embedder;
    private readonly IFaceDetector _detector; // wrap OpenCv detector
    private readonly GalleryRepository _gallery; // loads enrolled embeddings

    public RecognitionService(FaceEmbedder embedder, IFaceDetector detector, GalleryRepository gallery) {
        _embedder = embedder; _detector = detector; _gallery = gallery;
    }

    public List<FaceResult> Recognize(Mat image, float thr) {
        var boxes = _detector.Detect(image);
        var results = new List<FaceResult>();
        foreach (var box in boxes) {
            var face = CropAlign(image, box); // implement alignment
            var emb = _embedder.Embed(face);
            string bestName = "Unknown"; float bestDist = float.MaxValue;
            foreach (var item in _gallery.All()) {
                var dist = Similarity.Cosine(emb, item.Embedding);
                if (dist < bestDist) { bestDist = dist; bestName = item.Name; }
            }
            var conf = Similarity.Confidence(bestDist, thr);
            results.Add(new FaceResult { Box = box, Name = bestDist <= thr ? bestName : "Unknown", Confidence = conf });
        }
        return results;
    }
}

```

```

tName : "Unknown", Distance = bestDist, Confidence = conf });
    }
    return results;
}
}

```

Controller (upload → annotate → save → mark attendance)

```

[HttpPost("/attendance/upload")]
public async Task<IActionResult> Upload(IFormFile photo, float threshold = 0.40f) {
    if (photo is null || photo.Length == 0) return BadRequest("No file.");
    var fileName = $"cls_{DateTime.Now:yyyyMMdd_HH:mm:ss}.jpg";
    var path = Path.Combine(_env.WebRootPath, "uploads", fileName);
    using (var fs = System.IO.File.Create(path)) await photo.CopyToAsync(fs);

    using var img = Cv2.ImRead(path);
    var results = _recognition.Recognize(img, threshold);

    // draw & save annotated
    foreach (var r in results) {
        var color = r.Name == "Unknown" ? Scalar.Red : Scalar.Green;
        Cv2.Rectangle(img, r.Box, color, 2);
        Cv2.PutText(img, $"{r.Name} ({r.Confidence*100:0.0}%)", new Point(r.Box.X, Math.Max(20, r.Box.Y - 10)),
            HersheyFonts.HersheySimplex, 0.8, color, 2);
        if (r.Name != "Unknown") _attendance.Mark(r.Name, DateTime.Now, r.Confidence, fileName);
    }
    var outName = $"recognized_{DateTime.Now:yyyyMMdd_HH:mm:ss}.jpg";
    var outputPath = Path.Combine(_env.WebRootPath, "outputs", outName);
    Cv2.ImWrite(outputPath, img);

    return View("Result", new ResultVm { ImagePath = $"/outputs/{outName}",

```

```
Matches = results });  
}
```

📌 [Screenshot Placeholder: Result page with green/red boxes]

📌 [Screenshot Placeholder: Attendance grid view]

7.3 Variant B — Hybrid (ASP.NET + Python DeepFace via REST)

Why choose this: reuses your working Python logic with DeepFace (VGG-Face + cosine), minimal C# CV code.

How it works

1. ASP.NET receives the uploaded classroom photo and stores it under `wwwroot/uploads/`.
2. ASP.NET calls **Python API** (e.g., `POST /recognize`) with JSON containing the uploaded image path or sends the image bytes.
3. Python (your code) runs **DeepFace** detection + embeddings + matching (you already have this).
4. Python responds with **faces + boxes + name + confidence** and an **annotated image path**.
5. ASP.NET logs attendance into SQL Server and displays results.

Python service (FastAPI skeleton)

```
# fastapi_app.py  
from fastapi import FastAPI, UploadFile, File  
from pydantic import BaseModel  
from recognition import recognize_image # wrap your code  
app = FastAPI()  
  
class RecogResult(BaseModel):  
    name: str  
    confidence: float  
    box: dict # x,y,w,h
```



```
@app.post("/recognize")
async def recognize(file: UploadFile = File(...)):
    content = await file.read()
    # save temp, run your recognize_from_image → return list of results + annotated image path
    results, annotated_path = recognize_image(content)
    return {"results": results, "annotated_image": annotated_path}
```

ASP.NET calls Python

```
[HttpPost("/attendance/upload")]
public async Task<ActionResult> Upload(IFormFile photo) {
    // save file as before...
    using var ms = new MemoryStream();
    await photo.CopyToAsync(ms);
    var content = new MultipartFormDataContent {
        { new ByteArrayContent(ms.ToArray()), "file", photo.FileName }
    };
    var resp = await _http.PostAsync("http://python-host:8000/recognize", content);
    var payload = JsonSerializer.Deserialize<RecogPayload>(await resp.Content.ReadAsStringAsync());

    foreach (var r in payload.Results) {
        if (r.Name != "Unknown") _attendance.Mark(r.Name, DateTime.Now, r.Confidence, photo.FileName);
    }
    return View("Result", new ResultVm { ImagePath = payload.AnnotatedImage, Matches = payload.Results });
}
```

Pros

- Leverages your **existing DeepFace** pipeline.
- Quicker to ship.

Cons

- Two runtimes to deploy/manage.
- Network call overhead.

 [Screenshot Placeholder: Colab/Server-side Python service running]

 [Screenshot Placeholder: ASP.NET Result page consuming Python output]

8) Results & Analysis

- **Accuracy:** Good with frontal/clear faces; improves with multiple reference images per student.
- **Speed:** Variant A avoids network latency; Variant B depends on Python server performance.
- **Reliability:** Confidence threshold (e.g., 0.40) calibrates precision vs. recall.
- **Auditability:** Annotated images + per-face confidence are stored.

 [Placeholder: Chart – Confidence vs. Threshold]

 [Placeholder: Table – Matches (Name, Distance, Confidence)]

9) Advantages

- Saves time and reduces proxy.
 - Web-based → accessible, central control, role-based access.
 - Works with your existing Python stack (**Hybrid**) or as a single .NET app (**Pure**).
 - Easy data export and integration with ERP/LMS.
-

10) Limitations

- Requires **good lighting** and **reasonable resolution**.
 - Occlusions/masks and extreme angles reduce accuracy.
 - Larger galleries increase compute time unless embeddings are indexed (FAISS/Annoy—possible as a future add-on).
-

11) Future Scope

- **Live webcam** stream capture (periodic snapshots during lecture).
 - **Incremental learning**: add embeddings from new photos.
 - **Notifications** (email/WhatsApp) and **analytics** (attendance trends).
 - **Face quality checks** and **anti-spoofing** (liveness).
 - **GPU acceleration** (CUDA/TensorRT) for faster inference.
-

12) Conclusion

We implemented a robust, **web-based attendance system** on ASP.NET, offering two paths: a **pure C#** inference stack and a **hybrid** approach that reuses your proven Python/DeepFace pipeline. Both meet the objectives of speed, accuracy, and operational simplicity while leaving room for real-time extensions and enterprise integration.

13) Implementation & Deployment Guide (Quick Steps)

Variant A (Pure .NET)

1. Create **ASP.NET Core MVC** project.
2. Add NuGet packages; place ArcFace & detector **.onnx** models in `models/`.
3. Implement **FaceDetector**, **FaceEmbedder**, **RecognitionService** (as above).
4. Build **Enroll**, **Upload**, **Result**, **Reports** pages.
5. Configure **EF Core** for SQL Server; run migrations to create tables.
6. Deploy to **IIS** or **Azure App Service**.

Variant B (Hybrid)

1. Keep your **Python DeepFace** code; wrap with **FastAPI/Flask** endpoint.
2. Host Python service (local/VM/Docker).

3. ASP.NET calls Python API on upload; receives JSON with results + annotated image path.
 4. Same DB/UI flow as Variant A.
 5. Deploy ASP.NET and Python services; set API base URL via configuration.
-

14) References

- OpenCV / OpenCvSharp
 - Microsoft.ML.OnnxRuntime
 - InsightFace / ArcFace (ONNX)
 - FastAPI / Flask (for Hybrid)
 - ASP.NET Core docs
-

15) Screenshot Placeholders

- **[Enrollment Page]**
 - **[Upload/Capture Page]**
 - **[Annotated Result Image]**
 - **[Attendance Table / Report Export]**
 - **[Python Service (Hybrid) console]**
 - **[Database ER Diagram]**
-

Want this as a Word/PDF?

I can package this into a **clean Word (.docx)** with a cover page, headers/footers, and **boxed placeholders** for screenshots — or export to **PDF**.

Tell me which you prefer, and I'll generate it now.